

FactorMAD: A Multi-Agent Debate Framework Based on Large Language Models for Interpretable Stock Alpha Factor Mining

Yitong Duan
Tsinghua University
Beijing, China
dythuhu@foxmail.com

chuheng zhang
microsoft
Beijing, China
zhangchuheng123@live.com

Jian Li
Tsinghua University
Beijing, China
lijian83@mail.tsinghua.edu.cn

Abstract

In quantitative investment, alpha factor mining plays a crucial role in predicting stock returns. Traditional approaches rely on human experts to design factors based on financial intuition. To enhance the efficiency of factor mining, recent machine learning (ML) methods have driven a shift toward automated factor mining. However, these ML-based approaches often suffer from a lack of interpretability or are restricted by predefined mathematical operators, limiting their ability to mine novel and effective factors. In this paper, we propose FactorMAD, a multi-agent debate framework based on large language models (LLMs) for interpretable alpha factor mining. Unlike existing methods, our framework employs two specialized LLM agents that iteratively refine factors through structured debate, leveraging diverse prior perspectives and critiques. To enhance factor flexibility and expressiveness, we develop a code-based factor development pipeline that allows LLMs to directly produce implementable factor code instead of being restricted to predefined operator sets. Empirically, we conduct extensive experiments on real-world stock market data, demonstrating that FactorMAD significantly outperforms existing automated factor mining methods in predictive performance and trading profitability. Our findings highlight the potential of LLM-driven multi-agent debate as a powerful paradigm for mining high-quality alpha factors in financial markets.

Keywords

Large Language Model, Multi-agent Debate, Alpha Mining

ACM Reference Format:

Yitong Duan, chuheng zhang, and Jian Li. 2025. FactorMAD: A Multi-Agent Debate Framework Based on Large Language Models for Interpretable Stock Alpha Factor Mining. In *6th ACM International Conference on AI in Finance (ICAIF '25)*, November 15–18, 2025, Singapore, Singapore. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3768292.3770377>

1 Introduction

Factor models play an important role in quantitative investing. The core challenge of factor models is to mine effective alpha factors

that can forecast future stock trends. This task becomes increasingly important as markets become more efficient, necessitating the mining of adaptive factors that can effectively capture the complex dynamics of financial markets.

In traditional factor model research, factors are usually designed by human experts who leverage their financial expertise and market insights to capture market signals. These domain experts analyze historical market data, identify meaningful patterns, and construct factors through a combination of theoretical understanding and empirical observation [1, 8, 23]. However, this manual approach faces several limitations. First, the inefficiency of manual factor mining severely limits the number of factors that can be explored, making it challenging to mine factors efficiently in the face of the growing complexity and volume of financial data. Second, the subjective bias of human experts may constrain the factor exploration space, potentially overlooking novel and adaptive factors.

Recent advancements in machine learning (ML) introduce data-driven methods to automate factor mining. These methods can be broadly categorized into two types. The first type is implicit factor mining, which leverages neural networks to extract complex patterns directly from raw data, typically by identifying latent factors from hidden layer representations [5, 20]. However, these approaches often result in factors lacking explicit mathematical formulations, significantly limiting their interpretability. The lack of interpretability makes it difficult to understand the underlying mechanisms of the factors, hindering their application in real-world investment. The second type is explicit factor mining, which usually employs machine learning techniques such as genetic programming [3] or reinforcement learning [39] to generate explicit factor expressions from predefined operator sets. Although these methods produce interpretable factors with clear mathematical expressions, they are constrained by the limited expressiveness of predefined operators, making it challenging to mine effective and flexible factors for complex financial markets.

The recent emergence of large language models (LLMs) offers a promising perspective for tackling complex tasks in real-world applications, such as math [19, 33, 35], code [2, 12, 32], medicine [28, 43] and finance [16, 30], since LLMs excel at leveraging domain knowledge and generating creative solutions [11, 29]. Particularly, the multi-agent debate framework allows specialized agents to engage in iterative critique and refinement processes to optimize solutions. Many studies show that this structured debate approach substantially enhances reasoning capabilities, truthfulness, and evaluation accuracy [14, 34, 36]. Through interactions between agents, this framework enables more comprehensive exploration of the solution space, ultimately leading to more robust and reliable solutions.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICAIF '25, Singapore, Singapore

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2220-2/25/11
<https://doi.org/10.1145/3768292.3770377>

Building on these advancements, we introduce FactorMAD, a novel framework for interpretable factor mining based on multi-agent debate. Our framework employs two LLM agents with distinct prior knowledge, which engage in structured iterative debates to improve factors. During each debate round, the agents alternate between proposing and evaluating candidate factors and providing constructive critiques, continuously refining factor quality through this debate process. We empirically demonstrate that this alternating debate mechanism enables more effective exploration of the solution space compared to single-agent approaches that rely solely on self-improvement. Furthermore, we develop a code-based factor development pipeline that leverages LLMs' code generation capabilities to directly produce factor implementation code. This pipeline significantly enhances the flexibility and expressiveness of factor generation, facilitating the mining of more effective and interpretable factors.

In summary, this work makes the following key contributions:

- We propose FactorMAD, a multi-agent framework powered by large language models, where two LLM agents with distinct prior knowledge engage in structured debate interactions. Through iterative critique and refinement, our framework continuously improves factors, enabling the discovery of more effective predictive signals.
- We develop a novel factor development pipeline that takes advantage of LLMs' code generation capabilities. Unlike existing explicit factor mining methods, which are limited by predefined operator sets, our method directly generates standardized factor implementation code, significantly improving the flexibility and expressiveness of mined factors.
- Through extensive experiments on real-world market data, we demonstrate that our multi-agent debate framework consistently outperforms existing methods across multiple evaluation metrics. The results validate the effectiveness of our approach in generating high-quality interpretable factors for quantitative stock investment.

2 Related Work

2.1 Automated Factor Mining

Automated factor mining approaches can be broadly classified into two categories. The first category employs neural networks to extract hidden factors directly from raw market data [9, 37]. Gu et al. [10] introduce a conditional asset pricing model that leverages autoencoder neural networks to identify hidden factors from market data. Building upon this framework, Duan et al. [5] combine dynamic factor models with variational autoencoders to model noise components and estimate hidden factors. Lin et al. [20] reformulate risk factor mining as a representation learning problem, utilizing deep neural networks to generate meaningful risk factors. Duan et al. [6] adopt a hypergraph-based architecture with temporal residual contrastive learning to capture complex nonlinear relationships among hidden factors. While these neural approaches demonstrate strong empirical performance, they typically generate factors without explicit mathematical formulations, limiting their interpretability and making it challenging to understand the underlying mechanisms behind the mined factors.

The second category employs machine learning techniques to automatically synthesize factor expressions with clear and interpretable mathematical expressions [25, 38]. Zhang et al. [41] propose an evolutionary algorithm that combines hierarchical search strategies with quality diversity methods to generate formula-based factors. This approach demonstrates improved effectiveness in mining diverse and profitable factors compared to traditional methods. Building upon this, Cui et al. [3] introduce a factor mining framework based on auto machine learning, which not only generates new factors but also ensures they exhibit high returns while maintaining low correlations with existing factors. Further advancements in this direction include Yu et al. [39], who propose a reinforcement learning-based approach that optimizes factor mining by specifically focusing on synergistic alpha sets, leveraging the complementary nature of different factors to enhance overall performance. Shi et al. [24] introduce a generative-predictive neural network architecture for factor mining and combination, which simultaneously generates factors and dynamically adjusts their weights based on real-time performance metrics, enabling more adaptive factor strategies. With the recent emergence of large language models (LLMs), researchers have begun exploring their potential in factor mining [17, 31, 40]. Wang et al. [31] introduce a novel factor mining paradigm based on human-AI interaction, utilizing prompt engineering techniques to leverage LLMs for generating creative and effective factors. Furthermore, Yuan et al. [40] introduce a human-in-the-loop approach to factor mining, where human experts provide high-level guidance to LLMs, enabling more effective factor mining. Li et al. [17] develop an approach utilizing chain-of-thought reasoning to enable LLMs to iteratively mine and refine factors through structured logical steps. However, these explicit factor mining methods generate factor expressions based on predefined operators, limiting the flexibility and expressiveness of factors.

2.2 LLM-Based Multi-Agent Debate

LLMs demonstrate powerful reasoning and planning capabilities, proving highly effective in solving complex problems across diverse domains. A particularly promising advancement in LLM applications is multi-agent debate, which is used to enhance reasoning, truthfulness, and evaluation accuracy through structured debate among multiple agents [4, 15, 27]. Several pioneering works establish the theoretical foundations and practical applications of multi-agent debate frameworks. Estornell et al. [7] develop a theoretical framework for multi-agent debate, proposing novel interventions to improve debate efficacy and demonstrating significant performance improvements on some common benchmark tasks. Building upon this foundation, Chan et al. [2] introduce a multi-agent debate framework for autonomous text quality evaluation, leveraging LLMs with different role-specific prompts. Their approach achieves higher accuracy and better alignment with human assessments compared to single-agent methods. Further advancing the field, Khan et al. [14] demonstrate that structured debate consistently enhances both non-expert models' and humans' question-answering capabilities. Their study suggests that unsupervised optimization of expert debaters for persuasiveness improves non-experts' capacity to distinguish truth in debates. Liang et al. [18] explore how

multi-agent debate frameworks can encourage divergent thinking, thereby improving model robustness by exploring alternative perspectives and counterarguments. Smit et al. [26] conduct an in-depth examination of multi-agent debate strategies for optimizing debate-driven approaches.

Building upon these significant advancements in multi-agent debate frameworks, our work applies this powerful paradigm to factor mining. By employing LLMs as specialized debating agents with distinct prior knowledge, we create a structured framework for iteratively refining and optimizing factors through debate interactions, enabling the mining of more effective factors.

3 Methodology

In this section, we introduce FactorMAD, a novel framework for factor mining that leverages the multi-agent debate mechanism based on large language models. We begin by providing a formulation of the factor mining problem. Subsequently, we present a detailed introduction of our multi-agent debate framework. Finally, we introduce a code-based factor development pipeline, which serves as a crucial component of our multi-agent debate framework, handling the implementation details of each factor generated during the debate process.

3.1 Problem Formulation

In this paper, let $x \in \mathbb{R}^{N \times T \times D}$ denote the historical market data of stocks, and $y \in \mathbb{R}^{N \times T}$ represent the corresponding future returns, where N represents the number of stocks, T denotes the length of the historical period, and D indicates the dimension of market features for each stock. In our work, we use price and volume data as market features. A factor is defined as a function $f(\cdot)$ that maps the historical data x to a factor exposure vector $v_f \in \mathbb{R}^{N \times T}$, expressed as $v_f = f(x)$. To avoid look-ahead bias, $f(\cdot)$ must only use information available up to time t , meaning that $v_f^{(t)} = f(x_{\leq t})$ without accessing any future data $x_{>t}$. The predictive performance of the factor is evaluated using a metric $c = C(f(x), y)$, where $C(\cdot)$ represents an evaluation function such as the information coefficient (IC) between factor exposure values and future returns.

The objective of factor mining is to find a set of factors that can effectively predict future returns when combined. Formally, we aim to mine a set of M factors $\mathcal{F} = \{f_1, f_2, \dots, f_M\}$ to maximize the overall predictive performance:

$$\max_{\mathcal{F}} C(g(\mathcal{F}(x), \Phi), y), \quad (1)$$

where $g(\cdot)$ represents the combination function of the factors (e.g., a linear model), and Φ denotes the parameters of the combination function.

3.2 Agent Debate Framework

We propose a multi-agent debate framework for factor mining that consists of three key components: agent initialization, debate process, and convergence criteria. First, we initialize two LLM agents with distinct perspectives on factor design to ensure diverse viewpoints during the debate. Second, these agents engage in an iterative debate process where they alternate between critiquing each other's proposals and generating improved factors based on those critiques.

Finally, the debate continues until it meets predefined convergence criteria, enabling the exploration of the solution space. As illustrated in Figure 1, this structured debate framework leverages the complementary strengths of multiple agents to progressively enhance factor quality. The following sections provide detailed explanations of each component within our framework.

3.2.1 Agent Initialization. We initialize two agents with distinct perspectives before debating. This initialization adopts a stepback method that first extracts high-level insights from existing factors and then leverages these insights to establish distinct perspectives for the agents. Through this approach, agents extract different high-level principles and strategies before engaging in debate, ensuring that each agent contributes a distinct perspective to the debate. This diversity in initial knowledge enriches the debate process and establishes a robust foundation for mining effective factors.

In particular, we initialize the agents by randomly selecting several existing factors as few-shot examples. These examples serve as the foundation from which the agents can derive their respective initial perspectives and develop expertise in factor mining. Formally, let $\mathcal{A}_j (j \in \{0, 1\})$ denote the two agents involved in the debate. Each agent $\mathcal{A}_j$ is provided with a set of examples $\mathcal{E}_j = \{\alpha^{(j_1)}, \alpha^{(j_2)}, \dots, \alpha^{(j_k)}\}$, where $\alpha^{(k)} = (f^{(k)}, c^{(k)})$ represents the complete information of the k -th factor, including both the factor implementation code and its performance metrics. Inspired by Zheng et al. [42], rather than directly using these examples as prompts for the agents, we first analyze and summarize key features and valuable insights from the examples. This abstraction process helps agents develop higher-level understanding and principles about factor design. Formally, let $\mathcal{F}_e$ denote the set of existing factors. We randomly sample a subset of factors $\mathcal{E}_j \subseteq \mathcal{F}_e$ and then extract the key perspectives b_j for each agent $\mathcal{A}_j$ based on the sampled factors, b_j contains the high-level insights for factor design, such as data normalization, window smoothing, and so on. The detailed agent initialization process is formulated as follows:

$$\begin{aligned} \mathcal{E}_j &\sim \mathcal{P}(\mathcal{F}_e), \\ b_j &= \text{LLM}_{\text{sum}}(\mathcal{E}_j), \quad j \in \{0, 1\}, \\ \mathcal{A}_j &\leftarrow \text{LLM}(\pi_\alpha, b_j, \mathcal{E}_j), \end{aligned} \quad (2)$$

where $\mathcal{P}(\mathcal{F}_e)$ denotes the sampling distribution over the factor set $\mathcal{F}_e$, $\text{LLM}_{\text{sum}}(\cdot)$ leverages a large language model to extract key design principles b_j from the sampled factors $\mathcal{E}_j$. The function $\text{LLM}(\cdot)$ initializes each agent with its unique perspective derived from example sets, ensuring that agents bring diverse viewpoints to the debate process, and π_α represents the requirements of factor design.

3.2.2 Iterative Debate. Our framework implements a structured multi-round dialogue process where agents alternate between proposing candidate factors and providing critiques of each other's proposals, while simultaneously improving factors in response to the ongoing discourse.

The debate process is initialized with a seed factor, which serves as the basic factor for subsequent iterations between agents. To achieve a balance between effectiveness and diversity in factor improvement, we introduce a probabilistic seed factor generation mechanism. This mechanism allows an agent to either select from

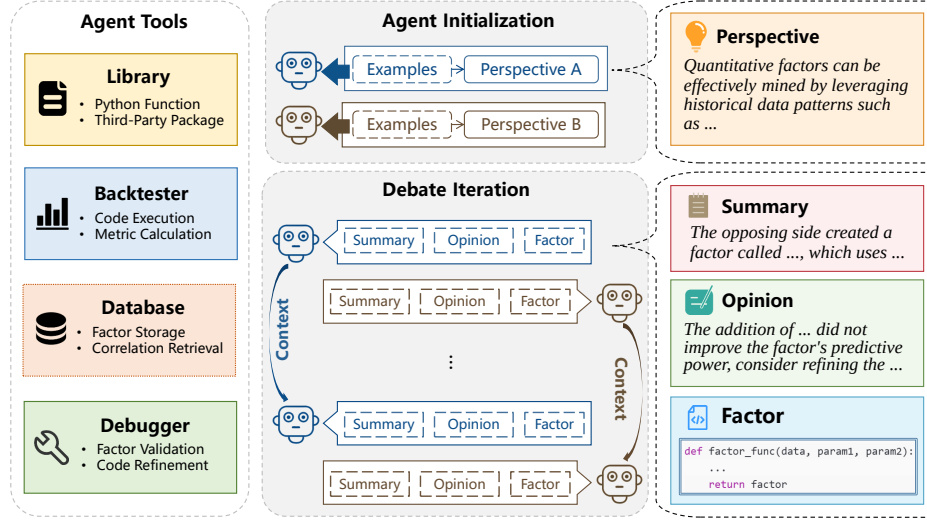


Figure 1: The overall multi-agent debate framework of FactorMAD for factor mining.

the factor set (leveraging existing factors) or generate a new factor (exploring innovative factors). Formally, let α_0 denote the seed factor, we have:

$$\alpha_0 \sim \begin{cases} \mathcal{P}(\mathcal{F}_e), & \text{w.p. } p_{\text{seed}} \\ \mathcal{A}_0(\pi_{\text{seed}}), & \text{w.p. } 1 - p_{\text{seed}} \end{cases}, \quad (3)$$

where $p_{\text{seed}} \in [0, 1]$ is the hyperparameter to control the probability of selecting an existing factor or generating a new factor, and π_{seed} represents the prompt guiding $\mathcal{A}_0$ in generating a new seed factor.

Given the seed factor α_0 , the debate proceeds through multiple rounds of structured interactions. Formally, at each round t , the improved factor α_t is generated based on the last factor α_{t-1} and the complete history of previously generated factors $\{\alpha_i\}_{i=0}^{t-2}$ in the debate:

$$\alpha_t \sim \mathcal{A}_j(\alpha_{t-1}, \{\alpha_i\}_{i=0}^{t-2}), \quad (4)$$

where $j = t \bmod 2$ denotes the agent responsible for proposing the factor at round t , alternating between the two agents to ensure diverse perspectives.

To obtain structured critiques and improvements during the debate, we design a content format for each debate round. This format ensures that agents engage in meaningful, focused interactions leading to continuous improvement of factors. Specifically, the structured debate content comprises three components:

- **Summary:** The agent provides a comprehensive analysis of the opponent's previous proposal, establishing a shared understanding of the factor's characteristics and underlying financial insights.
- **Critique:** The agent conducts a detailed critique of the opponent's proposal, identifying both strengths and potential areas for improvement in the factor design.
- **Factor:** Following the analytical critique, the agent generates an improved factor α_t through the factor development

pipeline (detailed in the following section). This factor incorporates the insights gained from the critique phase and serves as the new proposal for the next round of debate.

The debate process concludes when predefined stopping criteria are satisfied, such as a predefined maximum number of debate rounds. Throughout the entire debate process, all generated effective factors are recorded, evaluated, and stored in the factor set $\mathcal{F}_e$. The complete multi-agent debate framework for factor mining is summarized in Algorithm 1.

3.3 Factor Development Pipeline

The factor development pipeline is a critical component of our multi-agent debate framework that supports the implementation, validation, and evaluation of factors proposed during the debate process. This pipeline ensures that each proposed factor meets strict implementation standards before being added to the factor set.

By leveraging the powerful code generation capabilities of LLMs, we propose a code-based factor development pipeline that directly generates standardized factor implementation code with flexibility and expressiveness, which represents a significant advancement over existing explicit factor mining methods that are constrained by predefined operators. As illustrated in Figure 2, our pipeline consists of four key stages: generation, validation, correction, and evaluation. This structured pipeline ensures that each generated factor exhibits not only strong predictive power but also industry-standard implementation compliance.

3.3.1 Generation. In the generation phase, the agent leverages the summary and critiques from the current debate to generate factor function code in Python. To ensure consistency and maintainability, we design a set of requirements, which encompass input-output specifications, library dependencies, and reference examples. These requirements serve as guidelines to ensure that all generated code follows a consistent format, thereby streamlining subsequent validation and refinement processes. Formally, let f_i denote

Algorithm 1 FactorMAD

Require: Historical market data x , future returns y , factor set $\mathcal{F}_e$, factor requirements π_α , seed factor prompt π_{seed} , total debate rounds T , maximum correction iterations K , seed probability p_{seed} , thresholds ϵ_1, ϵ_2

- 1: **// Agent Initialization Phase**
- 2: **for** $j = 0$ to 1 **do**
- 3: $\mathcal{E}_j \sim \mathcal{P}(\mathcal{F}_e)$ // Sample factors
- 4: $b_j \leftarrow \text{LLM}_{\text{sum}}(\mathcal{E}_j)$ // Extract perspectives
- 5: $\mathcal{A}_j \leftarrow \text{LLM}(\pi_\alpha, b_j, \mathcal{E}_j)$ // Initialize debate agents
- 6: **end for**
- 7: $\mathcal{A}_c \leftarrow \text{LLM}(\pi_\alpha)$ // Initialize correction agent
- 8: $\alpha_0 \sim \begin{cases} \mathcal{P}(\mathcal{F}_e), & \text{with probability } p_{\text{seed}}, \\ \mathcal{A}_0(\pi_{\text{seed}}), & \text{with probability } 1 - p_{\text{seed}} \end{cases}$ // Generate initial seed factor
- 9:
- 10: **// Iterative Debate Process**
- 11: **for** $t = 1$ to T **do**
- 12: $f_t^{(0)} \sim \mathcal{A}_j(\alpha_{t-1} \mid \{\alpha_i\}_{i=0}^{t-2}), j = t \bmod 2$ // Generate factor code
- 13: $\pi_e = \text{Check}(f_t^{(0)}, x)$ // Validate factor code
- 14: **for** $k = 1$ to K **do**
- 15: **if** $\pi_e \neq \emptyset$ **then**
- 16: $f_t^{(k)} \sim \mathcal{A}_c(\pi_e, \{f_t^{(i)}\}_{i=0}^{k-1})$ // Refine factor code
- 17: $\pi_e = \text{Check}(f_t^{(k)}, x)$ // Re-validate factor code
- 18: **else**
- 19: $f_t \leftarrow f_t^{(k)}$
- 20: **break**
- 21: **end if**
- 22: **end for**
- 23: $c_t = C(f_t(x), y)$ // Evaluate factor performance
- 24: $\alpha_t = (f_t, c_t)$
- 25: **if** $c_t > \epsilon_1$ and $\text{Corr}(f_t, \mathcal{F}_e) < \epsilon_2$ **then**
- 26: $\mathcal{F}_e \leftarrow \mathcal{F}_e \cup \alpha_t$ // Update factor set
- 27: **end if**
- 28: **end for**

the factor function generated by agent $\mathcal{A}_j$ at debate round t , then $f_t \sim \mathcal{A}_j(\{\alpha_i\}_{i=0}^{t-1} \mid \pi_\alpha, b_j, \mathcal{E}_j)$, where the generation process is conditioned on the debate context $\{\alpha_i\}_{i=0}^{t-1}$, standardization requirements π_α , the agent's perspective b_j , and example factors $\mathcal{E}_j$.

3.3.2 Validation. In the validation phase, we implement a comprehensive verification process to ensure the generated factor code meets essential standards for factor implementation. This validation phase evaluates multiple critical aspects, such as error-free code execution, no look-ahead bias, time-series stationarity, and dimension consistency. The detailed validation criteria and corresponding suggestions are summarized in Table 1. Factors that successfully pass all validation checks proceed to subsequent phases, while those failing to meet any of the criteria are sent to the correction module for refinement. Formally, the validation process can be formalized as $\pi_e = \text{Check}(f_t, x)$, where π_e represents the error feedback from the validation module, and x denotes the historical market data.

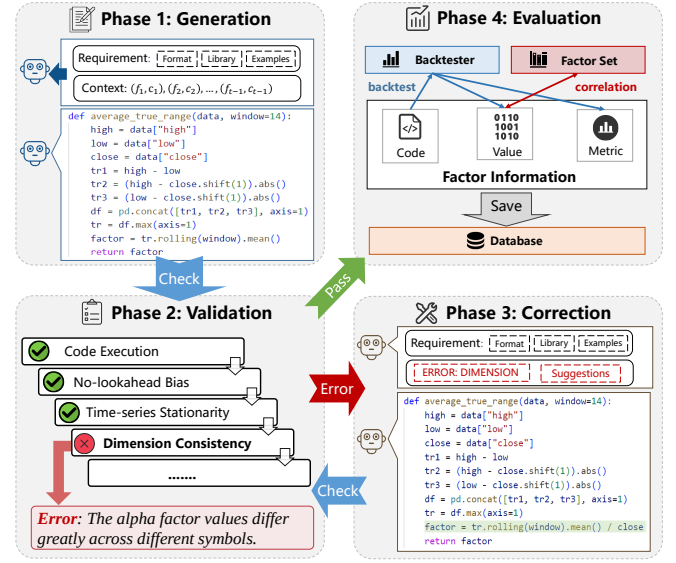


Figure 2: The workflow of the code-based factor development pipeline in FactorMAD. The pipeline consists of four key stages: generation, validation, correction, and evaluation.

3.3.3 Correction. In the correction phase, we employ a specialized LLM-based correction agent to refine factor codes that fail to meet the specified validation criteria. This agent operates through an iterative dialogue approach, identifying and addressing issues based on feedback from the validation module to ensure that generated factors meet comprehensive implementation standards. Formally, let $\mathcal{A}_c$ denote the correction agent and $f_t^{(k)}$ represent the factor code at correction iteration k . The correction process is defined as:

$$\begin{aligned} \mathcal{A}_c &\leftarrow \text{LLM}(\pi_\alpha), \\ f_t^{(k)} &\sim \mathcal{A}_c(\pi_e, \{f_t^{(i)}\}_{i=0}^{k-1}), \end{aligned} \quad (5)$$

where π_e represents the structured error feedback prompt from the validation module, and $\{f_t^{(i)}\}_{i=0}^{k-1}$ denotes the complete history of previous correction attempts. The refined code undergoes validation checks iteratively until it either passes all validation checks or reaches the predefined maximum number of correction iterations.

3.3.4 Evaluation. In the evaluation phase, we conduct a rigorous assessment of the corrected factor code to evaluate both its predictive performance and contribution to factor set diversity. The evaluation process focuses on two key aspects. First, we evaluate the factor's predictive performance by computing critical metrics such as information coefficient (IC) and information coefficient information ratio (ICIR). Formally, given the corrected factor code f_t at debate round t , we calculate the performance metrics $c_t = C(f_t(x), y)$, where $C(\cdot)$ represents the evaluation function, x denotes the historical market data, and y represents the future returns. Second, we analyze the factor's correlation with existing factors in the set $\mathcal{F}_e$ to ensure it contributes meaningful diversity to the overall factor set. Only factors that demonstrate both strong predictive power and low correlation with existing factors are added to the factor set as $\alpha_t = (f_t, c_t)$. This dual-criterion evaluation ensures that our

Table 1: The error status and corresponding suggestions in the factor validation phase for the FactorMAD framework. Once an error is detected, the corresponding suggestion is provided to guide the correction process.

Error	Suggestion
EXECUTION	You should carefully review the code to ensure there are no syntax errors.
IMPORT	Please make sure that the code does not import any third-party libraries in the function.
COMPLEX	Please simplify the calculation process of the function to reduce the number of lines of code.
TIME OUT	Please optimize the computational process of the factor function to speed it up.
DATA MISMATCH	Please ensure that only the data fields mentioned above are used in the factor calculation.
OUTPUT FORMAT	Please make sure that the return value of the function is a pandas Series.
OUTPUT NAN	Please check the calculation process in the function, like the division operation or the rolling window calculation.
LOOK-AHEAD BIAS	Please make sure not to include any data from future timestamps when calculating the factor value. 1. Use rolling functions instead of computing values over the entire time series, e.g., <code>x.rolling(window).mean()</code> instead of <code>x.mean()</code> .
DIMENSION	Ensure that factor values are dimensionless and comparable across different symbols. 1. Normalize price-related variables by dividing by the latest price or computing the rate of change. 2. Use rolling z-score normalization to standardize factor values.
ORDER	Ensure calculations in the factor function are independent of the order of symbols in the cross-section. 1. If using <code>groupby('time').apply()</code> , ensure only aggregation functions (mean, std, quantile) are used.
OTHER	Please carefully review the code for potential bugs and correct them.

framework continuously enriches the factor set with effective and diverse factors.

The evaluation results are recorded and used as valuable feedback for the debating agents, informing their subsequent factor proposals and critiques, thereby forming a closed-loop system that enables iterative factor improvement. This evaluation approach helps maintain the quality and diversity of our factor set while guiding the debate process toward more effective factor mining.

4 Experiments

In this section, we conduct extensive experiments to evaluate the effectiveness of FactorMAD in factor mining. Our experiments are designed to answer the following research questions:

- **RQ1:** How effectively do the factors mined through our framework predict future stock returns compared to state-of-the-art explicit factor mining approaches?
- **RQ2:** Can the factors generated by our framework be utilized to construct profitable trading strategies in the stock market?
- **RQ3:** Does the multi-agent debate mechanism enhance the quality of factors, and what are the advantages of this approach over non-debate methods?

4.1 Settings

4.1.1 Dataset. Our experiments are conducted on the China A-share market, including 5255 stocks listed on the Shanghai and Shenzhen stock exchanges. The dataset spans from 01/01/2014 to 12/31/2023, and is constructed using a sequence of market data, including daily high, open, low, and close prices, as well as volume-weighted average price (VWAP), total trading amount, and trading volume. Following standard practice in financial research, we employ a temporal split to partition the dataset: the in-sample period

(2014-2021) is utilized for factor mining and model training, while the out-of-sample period (2022-2023) serves as the out-of-sample dataset for evaluating the effectiveness of the mined factors.

In the factor mining task, the labels are the future stock returns over a 10-trading-day period, defined as $y_t = \text{VWAP}_{t+\Delta t+1} / \text{VWAP}_{t+1} - 1$, where $\Delta t = 10$, and y_t represents the future return at time t .

4.1.2 Baselines. In our experiments, we employ *GPT-4o 2024-08-06* as the base large language model in the multi-agent debate framework. To demonstrate the effectiveness of our approach in factor mining, we compare FactorMAD with several existing explicit factor mining methods. The baselines include:

- **GP** [21]: A genetic programming-based method that searches the combination of operators to generate factor expressions.
- **DSO** [22]: A deep symbolic optimization method that combines neural networks with symbolic regression to generate factors.
- **AlphaGen** [39]: A reinforcement learning-based approach that optimizes factor mining by focusing on synergistic factor sets.
- **CoE** [17]: An agent framework that leverages chain-of-experience reasoning to iteratively mine factors (based on *GPT-4o 2024-08-06*).

4.2 Stock Return Prediction

In this experiment, we evaluate the predictive performance of factors mined by different approaches. We employ three representative machine learning models:

- **Linear Regression (LR)**: A linear model widely used in traditional factor models due to its simplicity and interpretability, serving as a baseline model for evaluating the predictive power of mined factors.
- **Multi-Layer Perceptron (MLP)**: A 3-layer neural network that can model nonlinear patterns in the data, providing a more flexible and expressive alternative to the LR model.
- **LightGBM (LGB)** [13]: A gradient boosting framework known for its efficiency with structured data, capable of handling high-dimensional features and complex interactions.

These models are trained using the mean squared error (MSE) loss to predict future stock returns based on the mined factor sets. To ensure a thorough assessment of the predictive capabilities across different investment horizons, we evaluate performance using multiple prediction windows with future returns over $\Delta t = 10, 20, 30$ trading days as model labels. For each method, we mine 100 factors to serve as features for the models. Both features and labels undergo cross-sectional rank normalization preprocessing.

To evaluate the predictive performance of the factors, we report four key metrics based on the out-of-sample dataset. These metrics include the information coefficient (IC), which measures the correlation between signals and labels; the ranked information coefficient (RankIC), a more robust version of IC that compares ranked signals and labels; the mean squared error (MSE), which calculates the average squared difference between predicted and actual values; and the area under the curve (AUC), which assesses the model's ability to distinguish between positive and negative labels, where returns above zero are labeled as positive and those below zero as negative. The combination of these metrics provides a comprehensive evaluation of the mined factors. The summary of the overall experimental results is visualized in Figure 3.

The experimental results demonstrate that our proposed method exhibits superior predictive performance compared to other methods across all metrics and prediction horizons, indicating that the factors mined by our method are more effective in capturing predictive signals for predicting future stock returns.

4.3 Investment Simulation

To evaluate the practical profitability of the factors mined by different methods in real-world stock markets, we use the LR model as a linear factor model, as widely adopted in the industry, to conduct an investment simulation using a well-established stock selection strategy. This simulation employs the *TopK* strategy, which builds an equal-weighted portfolio that contains the k stocks with the highest predicted returns on each trading day, and this portfolio is held for Δt trading days. We adopt equal-weighted portfolios of CSI300 and CSI500 stocks, which are representative of large-cap and mid to small-cap stocks, as the benchmarks to evaluate the relative performance of our strategy.

In the simulation setup, we set $k = 50$ and $\Delta t = 10$, aligning with the prediction horizon of the mined factors. To ensure a realistic performance assessment, we set transaction costs at 0.3%.

Figure 4 presents the cumulative return (CR) curves for all methods compared over the out-of-sample period. For a more detailed

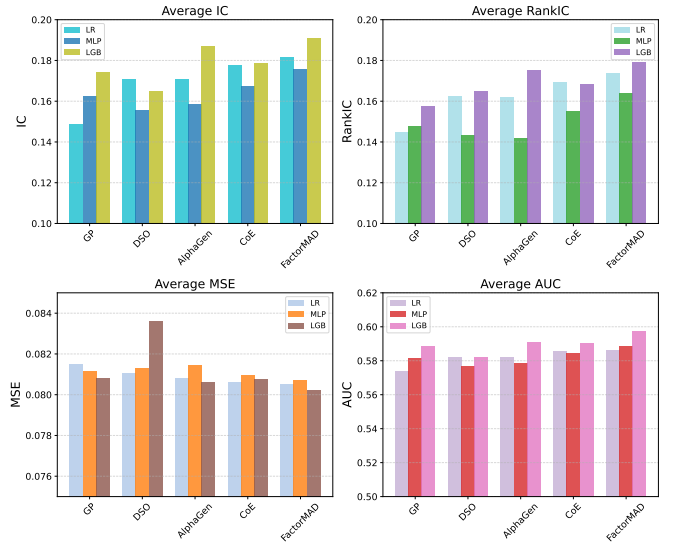


Figure 3: The average predictive performance of the LR, MLP, and LGB models. Higher values of IC, RankIC, and AUC indicate better performance, while lower MSE values are preferable.

performance assessment, Table 2 reports three key metrics calculated from the cumulative excess returns (CER) relative to the benchmark: the annualized return (AR), which measures the strategy's absolute performance; the information ratio (IR), which quantifies the strategy's risk-adjusted performance; and the return over maximum drawdown ratio (RoMaD), which also evaluates risk-adjusted returns by considering the worst peak-to-trough decline. The empirical results demonstrate that our debate-based approach achieves the highest performance, indicating that the factors mined by our method can be utilized to construct an effective factor model for investment.

Table 2: The investment simulation results of linear factor models based on factors mined by different methods. Higher values indicate better performance.

Methods	CSI300			CSI500		
	AR	IR	RoMaD	AR	IR	RoMaD
GP	0.004	0.175	0.047	-0.010	-0.449	-0.118
DSO	-0.011	-0.579	-0.123	0.070	2.727	1.860
AlphaGen	0.017	0.872	0.389	0.052	2.109	0.790
CoE	0.046	2.165	1.116	0.047	1.807	0.717
FactorMAD	0.063	2.984	1.350	0.086	3.411	1.341

4.4 Effectiveness of Multi-Agent Debate

In this section, we empirically evaluate the effectiveness of our multi-agent debate mechanism for factor mining through a comparative analysis.

To demonstrate the iterative improvement of our multi-agent debate framework, we compare FactorMAD with a single-agent

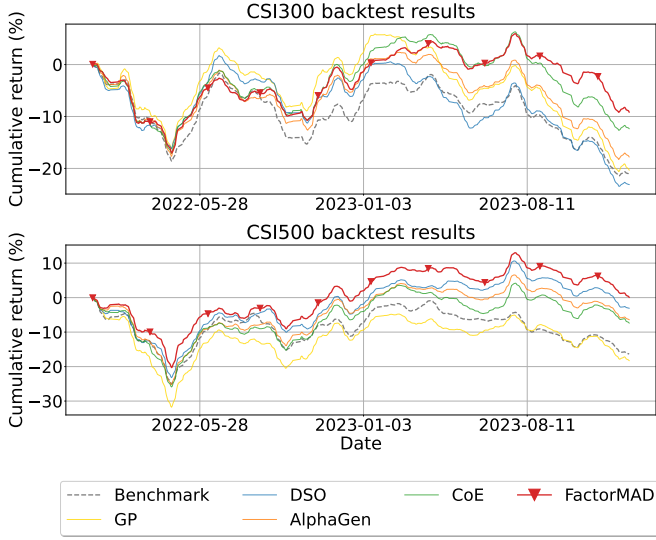


Figure 4: The cumulative return curves from simulated portfolio backtesting over out-of-sample period.

baseline approach. In this single-agent setting, only one agent is involved in the factor mining process without external critique, to improve the previously proposed factor by itself. Formally, the single-agent variant $\mathcal{A}'$ is defined as:

$$\begin{aligned} b' &= \text{LLM}_{\text{sum}}(\mathcal{E}_1 + \mathcal{E}_2), \\ \mathcal{A}'(x) &\leftarrow \text{LLM}(\pi_\alpha, b', \mathcal{E}_1 + \mathcal{E}_2), \end{aligned} \quad (6)$$

where example sets $\mathcal{E}_1$ and $\mathcal{E}_2$ are the same as those used in the multi-agent debate setting to ensure consistent initialization.

To ensure a fair comparison, we also maintain the same experimental settings, including the number of trials (100), iteration rounds (10), and agent initialization information. Figure 5 illustrates the average metrics of factors across iterations under both the multi-agent debate and single-agent mechanisms. The empirical results show that factors mined by the multi-agent debate framework exhibit significant improvements in the IC metric as the number of debate iterations increases. This enhancement in predictive capability suggests that the structured debate process effectively improves factor quality. In contrast, factors mined by the single-agent approach show limited performance gains across iterations. This significant difference in performance improvement provides strong empirical evidence for the superiority of our multi-agent debate framework.

Additionally, we examine the impact of the multi-agent debate mechanism on factor complexity by analyzing the code length of factors (measured by the number of non-comment lines in the implementation code). Figure 6 illustrates the evolution of factor complexity across iterations for both the multi-agent debate and single-agent approaches. The empirical results show that factors generated through the multi-agent debate framework consistently exhibit lower complexity than the single-agent baseline. This finding indicates that the debate process in our framework not only enhances factor performance but also promotes factor simplicity by eliminating redundant components. The reduction in factor

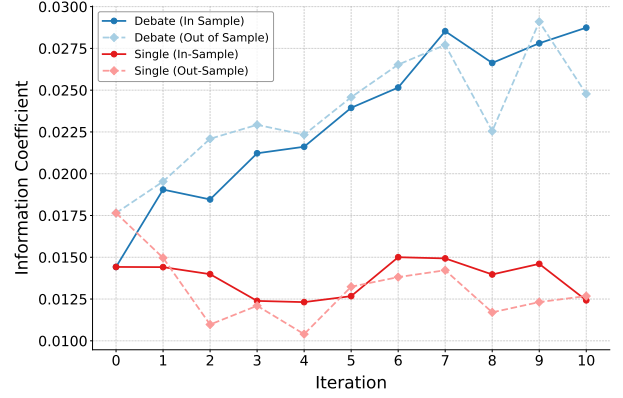


Figure 5: The metric improvement of factors over multi-agent debate iterations vs single agent iterations.

complexity is particularly beneficial for practical applications, as it enhances interpretability and reduces the risk of overfitting.

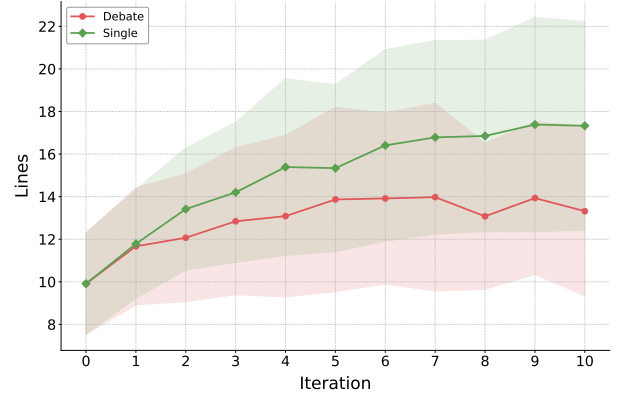


Figure 6: The evolution of factor complexity across iterations for the multi-agent debate and the single-agent approach. Solid lines represent the mean code length (measured by non-comment lines), while shaded regions indicate ± 1 standard deviation.

5 Conclusion

In this paper, we propose a novel multi-agent framework for factor mining, which leverages large language models to generate and improve stock factors through multi-agent debate. In our framework, two agents engage in a structured debate process to propose, critique, and refine factors, enabling the system to iteratively enhance factor quality. To enhance the flexibility of mined factors, we introduce a code-based factor development pipeline that directly produces factor codes meeting standardized requirements. Extensive experiments demonstrate that our framework achieves superior performance compared to existing methods. Additionally, the ablation study highlights the effectiveness of agent debate in enhancing the predictive power and reducing the complexity of the mined factors.

References

- [1] Mark M. Carhart. 1997. On Persistence in Mutual Fund Performance. *Journal of Finance* 52 (1997), 57–82. <https://api.semanticscholar.org/CorpusID:262814108>
- [2] Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. [n. d.]. Chateval: Towards better llm-based evaluators through multi-agent debate, 2023. URL <https://arxiv.org/abs/2308.07201> 3 ([n. d.]).
- [3] Can Cui, Wei Wang, Meihui Zhang, Gang Chen, Zhaojing Luo, and Beng Chin Ooi. 2021. Alphaevolve: A learning framework to discover novel alphas in quantitative investment. In *Proceedings of the 2021 International conference on management of data*. 2208–2216.
- [4] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. 2023. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first International Conference on Machine Learning*.
- [5] Yitong Duan, Lei Wang, Qizhong Zhang, and Jian Li. 2022. Factorvae: A probabilistic dynamic factor model based on variational autoencoder for predicting cross-sectional stock returns. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36. 4468–4476.
- [6] Yitong Duan, Weiran Wang, and Jian Li. 2025. FactorGCL: A Hypergraph-Based Factor Model with Temporal Residual Contrastive Learning for Stock Returns Prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 173–181.
- [7] Andrew Estornell and Yang Liu. 2024. Multi-LLM debate: Framework, principals, and interventions. *Advances in Neural Information Processing Systems* 37 (2024), 28938–28964.
- [8] Fama Eugene and Kenneth French. 1992. The cross-section of expected stock returns. *Journal of Finance* 47, 2 (1992), 427–465.
- [9] Achintya Gopal. 2024. Neuralfactors: A novel factor learning approach to generative modeling of equities. In *Proceedings of the 5th ACM International Conference on AI in Finance*. 99–107.
- [10] Shihao Gu, Bryan T. Kelly, and Dacheng Xiu. 2019. Autoencoder Asset Pricing Models. *SSRN Electronic Journal* (2019). doi:10.2139/ssrn.3335536
- [11] Shanshan Han, Qifan Zhang, Yuhang Yao, Weizhao Jin, Zhaozhao Xu, and Chaoyang He. 2024. LLM Multi-Agent Systems: Challenges and Open Problems. *CoRR* abs/2402.03578 (2024). arXiv:2402.03578 doi:10.48550/ARXIV.2402.03578
- [12] Yoichi Ishibashi and Yoshimasa Nishimura. 2024. Self-Organized Agents: A LLM Multi-Agent Framework toward Ultra Large-Scale Code Generation and Optimization. *CoRR* abs/2404.02183 (2024). arXiv:2404.02183 doi:10.48550/ARXIV.2404.02183
- [13] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems* 30 (2017).
- [14] Akbir Khan, John Hughes, Dan Valentine, Laura Ruis, Kshitij Sachan, Ansh Radhakrishnan, Edward Grefenstette, Samuel R Bowman, Tim Rocktäschel, and Ethan Perez. 2024. Debating with more persuasive llms leads to more truthful answers. *arXiv preprint arXiv:2402.06782* (2024).
- [15] Kyungha Kim, Sangyun Lee, Kung-Hsiang Huang, Hou Pong Chan, Manling Li, and Heng Ji. 2024. Can llms produce faithful explanations for fact-checking? towards faithful explainable fact-checking via multi-agent debate. *arXiv preprint arXiv:2402.07401* (2024).
- [16] Yang Li, Yangyang Yu, Haohang Li, Z. Chen, and Khaldoun Khashanah. 2023. TradingGPT: Multi-Agent System with Layered Memory and Distinct Characters for Enhanced Financial Trading Performance. <https://api.semanticscholar.org/CorpusID:261582775>
- [17] Zhiwei Li, Ran Song, Caihong Sun, Wei Xu, Zhengtao Yu, and Ji-Rong Wen. 2024. Can Large Language Models Mine Interpretable Financial Factors More Effectively? A Neural-Symbolic Factor Mining Agent Model. In *Findings of the Association for Computational Linguistics ACL*. 3891–3902.
- [18] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. 2023. Encouraging divergent thinking in large language models through multi-agent debate. *arXiv preprint arXiv:2305.19118* (2023).
- [19] Haoran Liao, Qinyi Du, Shaohua Hu, Hao HE, Yanyan Xu, Jidong Tian, and Yaohui Jin. 2024. Modeling Complex Mathematical Reasoning via Large Language Model based MathAgent. <https://openreview.net/forum?id=Ge7ZqrKG9t>
- [20] Hengxu Lin, Dong Zhou, Weiqing Liu, and Jiang Bian. 2021. Deep risk model: a deep learning solution for mining latent risk factors to improve covariance matrix estimation. In *ICAIF '21: 2nd ACM International Conference on AI in Finance, Virtual Event, November 3 - 5, 2021*, Anisoara Calinescu and Lukasz Szpruch (Eds.). ACM, 12:1–12:8. doi:10.1145/3490354.3494377
- [21] Xiaoming Lin, Ye Chen, Ziyu Li, and Kang He. 2019. *Stock Alpha Mining Based On Genetic Algorithm*. Technical Report. Huatai Securities Research Center. <https://crm.htsc.com.cn/doc/2019/10750101/f75b4b6a-2bdd-4694-b696-4c62528791ea.pdf>
- [22] Brenden K Petersen, Mikel Landajuela, T Nathan Mundhenk, Claudio P Santiago, Soo K Kim, and Joanne T Kim. 2019. Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. *arXiv preprint arXiv:1912.04871* (2019).
- [23] Lubos Pástor and Robert F. Stambaugh. 2001. Liquidity Risk and Expected Stock Returns. *SSRN Electronic Journal* (2001). doi:10.2139/ssrn.279804
- [24] Hao Shi, Weili Song, Xinting Zhang, Jiahe Shi, Cuicui Luo, Xiang Ao, Hamid Arian, and Luis Angel Seco. 2025. Alphaforge: A framework to mine and dynamically combine formulaic alpha factors. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 12524–12532.
- [25] Hong-Gi Shin, Sukhyun Jeong, Eui-Yeon Kim, Sungho Hong, Young-Jin Cho, and Yong-Hoon Choi. 2024. Synergistic Formulaic Alpha Generation for Quantitative Trading based on Reinforcement Learning. In *2024 International Conference on Information Networking (ICOIN)*. IEEE, 42–46.
- [26] Andries Smit, Paul Duckworth, Nathan Grinsztajn, Thomas D Barrett, and Arnu Pretorius. [n. d.]. Should we be going mad? A look at multi-agent debate strategies for llms (2024). URL <https://arxiv.org/abs/2311.17371> ([n. d.]).
- [27] Vighnesh Subramaniam, Antonio Torralba, and Shuang Li. 2024. DebateGPT: Fine-tuning Large Language Models with Multi-agent Debate Supervision. <https://openreview.net/forum?id=ChNy95ovpF>
- [28] Xiangru Tang, Anni Zou, Zhuosheng Zhang, Ziming Li, Yilun Zhao, Xingyao Zhang, Arman Cohan, and Mark Gerstein. 2023. Medagents: Large language models as collaborators for zero-shot medical reasoning. *arXiv preprint arXiv:2311.10537* (2023).
- [29] Qinqing Wang, Zihao Wang, Ying Su, Hanghang Tong, and Yangqiu Song. 2024. Rethinking the Bounds of LLM Reasoning: Are Multi-Agent Discussions the Key?. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2024, Bangkok, Thailand, August 11-16, 2024, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 6106–6131. doi:10.18653/V1/2024.ACL-LONG.331
- [30] Saizhuo Wang, Hang Yuan, Lionel M Ni, and Jian Guo. 2024. Quantagent: Seeking holy grail in trading by self-improving large language model. *arXiv preprint arXiv:2402.03755* (2024).
- [31] Saizhuo Wang, Hang Yuan, Leon Zhou, Lionel M Ni, Heung-Yeung Shum, and Jian Guo. 2023. Alpha-gpt: Human-ai interactive alpha mining for quantitative investment. *arXiv preprint arXiv:2308.00016* (2023).
- [32] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable Code Actions Elicit Better LLM Agents. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. <https://openreview.net/forum?id=j9BoXAIfa>
- [33] Yiran Wu, Feiran Jia, Shaokun Zhang, Han-Tai Li, Erkang Zhu, Yue Wang, Yin Tat Lee, Richard Peng, Qingyun Wu, and Chi Wang. 2023. MathChat: Converse to Tackle Challenging Math Problems with LLM Agents. <https://api.semanticscholar.org/CorpusID:259063798>
- [34] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. 2025. The rise and potential of large language model based agents: A survey. *Science China Information Sciences* 68, 2 (2025), 121101.
- [35] Wenbei Xie, Donglin Liu, Haoran Yan, Wenjie Wu, and Zongyang Liu. 2024. MathLearner: A Large Language Model Agent Framework for Learning to Solve Mathematical Problems. *CoRR* abs/2408.01779 (2024). arXiv:2408.01779 doi:10.48550/ARXIV.2408.01779
- [36] Kai Xiong, Xiao Ding, Yixin Cao, Ting Liu, and Bing Qin. 2023. Examining inter-consistency of large language models collaboration: An in-depth analysis via debate. *arXiv preprint arXiv:2305.11595* (2023).
- [37] Wentao Xu, Weiqing Liu, Lewen Wang, Yingce Xia, Jiang Bian, Jian Yin, and Tie-Yan Liu. 2021. HIST: A Graph-based Framework for Stock Trend Forecasting via Mining Concept-Oriented Shared Information. *CoRR* abs/2110.13716 (2021). arXiv:2110.13716 <https://arxiv.org/abs/2110.13716>
- [38] Wenyan Xu, Rundong Wang, Chen Li, Yonghong Hu, and Zhonghua Lu. 2025. HRFT: Mining High-Frequency Risk Factor Collections End-to-End via Transformer. In *Companion Proceedings of the ACM on Web Conference 2025*. 538–547.
- [39] Shuo Yu, Hongyan Xue, Xiang Ao, Feiyang Pan, Jia He, Dandan Tu, and Qing He. 2023. Generating synergistic formulaic alpha collections via reinforcement learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5476–5486.
- [40] Hang Yuan, Saizhuo Wang, and Jian Guo. 2024. Alpha-GPT 2.0: Human-in-the-Loop AI for quantitative investment. *arXiv preprint arXiv:2402.09746* (2024).
- [41] T. Zhang, Yuanqi Li, Yifei Jin, and Jian Li. 2020. AutoAlpha: an Efficient Hierarchical Evolutionary Algorithm for Mining Alpha Factors in Quantitative Investment. *arXiv: Computational Finance* (2020). <https://api.semanticscholar.org/CorpusID:211171490>
- [42] Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H Chi, Quoc V Le, and Denny Zhou. 2023. Take a step back: Evoking reasoning via abstraction in large language models. *arXiv preprint arXiv:2310.06117* (2023).
- [43] Hongjian Zhou, Fenglin Liu, Boyang Gu, Xinyu Zou, Jinfa Huang, Jing Wu, Yiru Li, Sam S Chen, Peilin Zhou, Junling Liu, et al. 2023. A survey of large language models in medicine: Progress, application, and challenge. *arXiv preprint arXiv:2311.05112* (2023).